Identificação de Componentes STM32MP157D

https://gemini.google.com/app/1dce103d8c1599f8

User prompt: quero q vc leia essas fotos, elas são dos componenetes pro projeto usando stm32, é uma caixa d algum modleo da strm32 e um sensor infravermelho, mas nn sei quais são os modelos/especificações, só tenho essas fotos

Response: Pelas etiquetas e detalhes visíveis nas fotos, você tem em mãos um **kit de desenvolvimento da linha de microprocessadores (MPU) STM32MP1** e um **módulo sensor de obstáculo infravermelho reflexivo de 3 pinos (padrão FC-51 / LM393)**.

1. Placa / Kit STM32 (Fotos 2 e 3)

[RichListCard: 3]

O STMicroelectronics STM32MP157D-DK1 é um **kit Discovery** com microprocessador híbrido **Dual Arm Cortex-A7 (800 MHz)** e **Cortex-M4 (209 MHz)** para aplicações com **Linux embarcado** e tempo real.

O que está escrito nas etiquetas da caixa

- **Modelo exato (CPN):** `STM32MP157D-DK1`.
- **Etiqueta de conferência (topo da caixa):** `KIT STM32MP1 DK1`, indicando que a caixa acompanha `CABO USB C - USB C` e `SD CARD` (com visto manual `OK`).
- **Código Finished Good (FG):** `DK32MP157D1$AU2`.
- **Controle de Qualidade (QA n°):** `Z2A42001` (Código de barras: `DK32MP157D1$AU2Z2A42001B`).

Especificações Técnicas (`STM32MP157D-DK1`)

- **Arquitetura Híbrida (MPU, não apenas MCU):** Diferente de um STM32 comum (como F1/F4), o chip **STM32MP157DAC1** combina um processador de aplicação e um microcontrolador no mesmo encapsulamento:
  - **2x Arm Cortex-A7 (32 bits @ 800 MHz):** Roda um sistema operacional **Linux completo** (como a distribuição oficial **OpenSTLinux** gravada no cartão microSD).
  - **1x Arm Cortex-M4 (32 bits @ 209 MHz com FPU/MPU):** Dedicado a tarefas de **tempo real** e leitura de sensores de baixa latência (programável via **STM32CubeIDE**), comunicando-se com o Linux via **OpenAMP / RPMsg**.
- **Memória e Armazenamento: 512 MB (4-Gbit) de RAM DDR3L** (16 bits @ 533 MHz) e slot para **cartão microSD** (usado para o boot do sistema operacional).
- **Conectividade e Interfaces de Placa:**
  - **Rede:** 1x porta **Gigabit Ethernet RJ45** (RGMII).
  - **USB:** 4x portas **USB 2.0 Host Tipo-A** + 1x porta **USB Tipo-C DRP (OTG)**.
  - **Vídeo e Áudio:** 1x saída **HDMI**, 1x interface de display **MIPI DSI** e codec de áudio com conector **P2 estéreo** (fone + microfone analógico).
  - **Depuração On-board: ST-LINK/V2-1** integrado (porta Micro-B dedicada para debug e Virtual COM port).
  - **Expansão de Pinos (GPIOs):** Conectores fêmea no padrão **Arduino Uno V3** e barra de **40 pinos GPIO no padrão Raspberry Pi** (nível lógico de **3,3 V**).
- **Alimentação:** Requer fonte **5 V / 3 A via conector USB Tipo-C**.

2. Sensor Infravermelho (Fotos 1 e 2)

[RichListCard: 11]

O Sensor de Obstáculo Reflexivo Infravermelho Ir Lm393 Arduino [Código 136] é um módulo detector de proximidade óptico com **alcance ajustável de 2 a 30 cm** e **saída digital** para microcontroladores.

O que está escrito na etiqueta da embalagem

- **Descrição:** `Sensor De Obstáculo Infravermelho Reflexivo Arduino Raspberry`.
- **SKU de estoque:** `SS10`.
- **Código de barras:** `TAPQ74883`.
- **Identificação visual na embalagem:** Através do plástico antiestático, é possível ver o **trimpot quadrado azul** de ajuste e os **dois LEDs de 5 mm** na ponta superior, além dos **3 pinos de conexão** na base inferior, confirmando o módulo clássico **FC-51 / HW-201 com CI LM393**.

Especificações Técnicas (Módulo IR Reflexivo FC-51 / LM393)

- **Tensão de Operação (`VCC`): 3,3 V a 5 V DC**.
  - *Dica de segurança para o seu STM32:* Alimente o sensor pelo pino de **3,3 V** da placa `STM32MP157D-DK1` (e não em 5 V). Como a saída `OUT` acompanha a tensão de `VCC`, alimentar com **3,3 V** protege as portas GPIO do STM32.

- **Faixa de Detecção: 2 cm a 30 cm** (ajustável girando o parafuso do potenciômetro azul na placa), com ângulo de detecção de aproximadamente **35°**.
- **Circuito Integrado:** Comparador de tensão **LM393**.
- **Pinagem (3 Pinos):**
  - `VCC`**:** Alimentação positiva (3,3 V).
  - `GND`**:** Terra comum com a placa.
  - `OUT`**:** Saída de sinal **Digital (0 ou 1)**:
    - **Sem obstáculo:** Saída em nível lógico **ALTO (**`HIGH` **/ 3,3 V)**.
    - **Obstáculo detectado:** Saída vai para nível lógico **BAIXO (**`LOW` **/ 0 V)** e o LED indicador na placa do sensor acende.

Se quiser, posso montar o **esquema de ligação pino a pino** desse sensor nos conectores GPIO/Arduino da sua **STM32MP157D-DK1** e gerar um exemplo de código (seja em **Python/C++ no Linux** pelo Cortex-A7 ou via **HAL no STM32CubeIDE** pelo Cortex-M4) para ler o sensor. Você pretende rodar a lógica principal no Linux da placa ou no coprocessador Cortex-M4?

User prompt: é para fazer esse projeto, quero q vc faça um roadmap, considerando q as datas estão erradas, na verdade a entrega 0 vai ser dia 19/10 e cada entrega depois vai ser na proxima segunda, então a entrega 1 vai er dia 26/10, entrega 2 dia 2/11 etc

Response: Aqui está o **Roadmap completo do Projeto de Programação Aplicada (Monitoramento Inteligente de Carga)**, ajustado para o seu **Sensor Nº 3 — Módulo de Obstáculo Infravermelho Reflexivo Digital (HW-201)** na placa **STM32MP157D-DK1**, com o cronograma corrigido para entregas semanais às segundas-feiras a partir do dia **19/10/2026**.

🗓️ Visão Geral do Cronograma Corrigido

Etapa	Data (Segunda-feira)	Pontuação	Resumo do Entregável
Entrega 0	**19/10/2026**	Obrigatório	Foto da conexão física do sensor com o kit **desligado**.
Entrega 1	**26/10/2026**	**2,0 pts**	Classe em C++ para leitura do sensor documentada com Doxygen.
Entrega 2	**02/11/2026**	**2,0 pts**	Executável rodando no kit (cross-compilado) + `README.md` com instruções.
Entrega 3	**09/11/2026**	**2,0 pts**	Envio contínuo das leituras via socket UDP + documentação do protocolo.
Entrega 4	**16/11/2026**	**2,0 pts**	Interface Gráfica (GUI) no PC com tempo real, gráfico (30–60 s) e log CSV.
Entrega 5	**23/11/2026**	**2,0 pts**	Vídeo de demonstração, apresentação, diagramas no `README.md` e integração.

🛠️ Detalhamento Técnico por Etapa (Passo a Passo)

📍 Fase 0: Montagem Segura e Preparação do Ambiente (Hoje, 15/10 até 19/10)

- **Objetivo:** Garantir a ligação elétrica correta sem risco de queimar a placa e preparar o ambiente de compilação cruzada.
- **Atenção crítica de hardware:** O kit **jamais deve ser alimentado com 5V nos pinos de sensor**; utilize exclusivamente **3,3V** e **não energize a placa antes de enviar a foto ao professor**.
- **Checklist de Tarefas:**
  1. **Conectar o sensor HW-201 com o kit totalmente desligado** seguindo exatamente a **Figura 5** do guia:
     - `VCC` **do sensor (fio vermelho):** Pino **4** do conector de alimentação (`3V3`).
     - `GND` **do sensor (fio marrom/preto):** Pino **7** do conector de alimentação (`GND`).
     - `OUT / D0` **do sensor (fio laranja):** Pino **5** do conector analógico ao lado (`SINAL A0/D0`).
  2. **Tirar a foto nítida da montagem** (mostrando os pinos no header da STM32MP1 e no sensor com o cabo USB-C desconectado) e submeter na **Entrega 0**.
  3. **Criar o repositório Git** do grupo e estruturar o `README.md` inicial com a descrição do projeto e dependências.
  4. **Configurar o ambiente Linux e a Toolchain:** Baixar e extrair o pacote `arm-buildroot-linux-gnueabihf_sdk-DK2.tar.gz` (`tar -xvf arm-buildroot-linux-gnueabihf_sdk-DK2.tar.gz`) para habilitar a compilação cruzada.

📍 Fase 1: Modelagem POO, Mapeamento IIO e Doxygen (20/10 até 26/10)

- **Objetivo:** Implementar a classe C++ encapsulada para ler e interpretar o sensor de obstáculo IR, documentada com Doxygen.
- **Detalhe importante do PDF sobre a leitura:** Embora o módulo HW-201 seja digital, o guia padroniza a conexão de sensores analógicos e digitais na porta do conversor A/D, lendo os arquivos do subsistema IIO do Linux em `/sys/bus/iio/devices/iio:device0/in_voltageX_raw`.
- **Checklist de Tarefas:**

1. **Descobrir o canal exato do ADC (após autorização da Entrega 0):** Conectar o pino de sinal ao `GND` com um jumper (conforme a Figura 3), acessar a placa via Serial (`115200 8N1`, usuário `root`, senha `root`) e testar `cat /sys/bus/iio/devices/iio:device0/in_voltageX_raw` até achar o canal que retorna `0`.
2. **Projetar a classe C++ (ex.: `IrObstacleSensor`):**
    - **Regra eliminatória:** É proibido usar variáveis globais ou funções soltas fora de classes.
    - Encapsular atributos privados (caminho do arquivo `sysfs`, último valor bruto lido, tensão convertida, estado booleano de violação/abertura do compartimento e limiar de detecção).
    - Implementar métodos públicos claros (ex.: `readRaw()`, `isCompartmentViolated()`, `getFormattedStatus()`).
3. **Documentar com Doxygen:** Inserir blocos de comentários `/** ... */` (`@brief`, `@param`, `@return`) em todos os arquivos `.hpp` e `.cpp` e validar a geração da documentação.
4. **Commit da Entrega 1:** Enviar o código e atualizar o `README.md` com a explicação da classe e seus métodos.

📍 **Fase 2: Cross-Compilação, Deploy e Testes na Placa (27/10 até 02/11)**

- **Objetivo:** Ter o binário rodando diretamente no Linux embarcado da STM32MP1-DK1, lendo o sensor fisicamente e exibindo os dados.
- **Checklist de Tarefas:**
    1. **Criar um `Makefile` ou `CMakeLists.txt`** apontando para o compilador `arm-buildroot-linux-gnueabihf-g++` e para o `sysroot` da toolchain fornecida.
    2. **Configurar a rede entre PC e Kit:** Conectar o cabo Ethernet, checar o IP da placa com `ifconfig` (padrão `192.168.42.2`) e fixar o IP do PC na mesma sub-rede (ex.: `192.168.42.10`).
    3. **Transferir e testar o binário:** Enviar o executável compilado via `scp <arquivo_local> root@192.168.42.2:/home/root` e executá-lo via SSH (`ssh root@192.168.42.2`).
    4. **Calibrar o sensor HW-201:** Ajustar o trimpot do sensor para a distância ideal de detecção de abertura/aproximação da carga e verificar os valores lidos no terminal (nível lógico baixo quando há obstáculo/reflexão e nível alto quando livre).
    5. **Atualizar o `README.md` (Entrega 2):** Documentar os comandos exatos de compilação cruzada, transferência via `scp`, execução na placa e adicionar **prints do terminal** mostrando o programa em funcionamento.

📍 **Fase 3: Comunicação em Tempo Real via Socket UDP (03/11 até 09/11)**

- **Objetivo:** Transmitir continuamente os dados lidos do sensor na STM32MP1 para o servidor central (PC) via rede local usando UDP.
- **Checklist de Tarefas:**
    1. **Modelar a comunicação dentro do padrão POO:** Criar uma classe `UdpTransmitter` (ou integrar de forma limpa à arquitetura orientada a objetos, mantendo zero variáveis globais) usando sockets POSIX (`<sys/socket.h>`, `<arpa/inet.h>`).
    2. **Definir e documentar o protocolo de aplicação:** Estruturar uma mensagem leve em **JSON** ou **CSV** contendo obrigatoriamente:
        - Identificador do sensor/grupo (ex.: `"sensor_id": "G3_IR_HW201"`);
        - Valor atual interpretado (ex.: `0` = Normal / `1` = Violação ou tensão medida);
        - Unidade (ex.: `"boolean"` ou `"V"`);
        - Timestamp (recomendado/bônus para já adiantar a Entrega 4).
    3. **Validar a transmissão na rede local:** Executar o cliente C++ na placa enviando pacotes para `192.168.42.10:<porta>` e testar a recepção no PC (usando `netcat -ul <porta>` ou um script receptor simples).
    4. **Atualizar o `README.md` (Entrega 3):** Incluir a especificação formal do protocolo UDP (formato dos campos, porta utilizada, frequência de envio) e prints do envio/recebimento no terminal.

📍 **Fase 4: Interface Gráfica de Monitoramento no Servidor (10/11 até 16/11)**

- **Objetivo:** Construir o dashboard no PC para receber os pacotes UDP e exibir o monitoramento da carga em tempo real.
- **Sugestão de Stack: Python com PyQt + Matplotlib (ou PyQtGraph)** permite criar rapidamente uma interface moderna, com thread ou `QUdpSocket` para não travar a tela e plotagem fluida em tempo real.
- **Checklist de Tarefas:**
    1. **Implementar os Requisitos Obrigatórios da GUI:**
        - Exibição em destaque do **valor/estado atual do sensor** (ex.: "Carga Íntegra" vs. "Violação Detectada");
        - **Gráfico de histórico temporal** com janela deslizante dos **últimos 30 a 60 segundos**;
        - **Alerta visual** (mudança de cor para vermelho/banner piscante) quando a leitura ultrapassar o limite pré-definido;
        - Botão/função para **salvar o histórico de dados em arquivo de log ou `.csv`**.
    2. **Implementar os Requisitos Recomendados (Garantir Bônus):**

- Campo na tela para **configuração dinâmica do limiar de alerta**;
- **Exportação automática** de relatórios;
- Exibição do **timestamp** de cada pacote e **indicador temporal da última atualização recebida** (ex.: *"Último pacote há 0.2s"*).

3. **Atualizar o `README.md` (Entrega 4):** Adicionar instruções de instalação/execução da interface gráfica e **capturas de tela (prints)** da GUI em operação normal e em estado de alerta.

📍 **Fase 5: Documentação Final, Vídeo e Integração Geral (17/11 até 23/11)**

- **Objetivo:** Consolidar toda a documentação exigida no `README.md`, gravar o vídeo de demonstração e preparar o código para a simulação prática integrada com os demais grupos.
- **Checklist de Tarefas:**
  1. **Completar todos os itens finais obrigatórios do `README.md`:**
     - **Fluxograma** de funcionamento do sistema embarcado e do servidor;
     - **Diagrama de Classes UML** mostrando o encapsulamento e relacionamento das classes C++;
     - **Descrição física e princípio de funcionamento do sensor HW-201** (emissão/reflexão IR e detecção de abertura do compartimento);
     - **Análise crítica dos valores medidos** (comportamento das leituras com compartimento fechado vs. aberto, alcance prático e resposta temporal);
     - Revisão das instruções de compilação/uso, documentação Doxygen, protocolo UDP e prints de todas as etapas.
  2. **Gravar o Vídeo de Demonstração e montar a Apresentação:** Mostrar o hardware montado na STM32MP1-DK1, o código rodando na placa, a atuação física no sensor infravermelho simulando uma violação de carga e a reação imediata na interface gráfica com gravação de log.
  3. **Preparação para a Simulação Prática Integrada:** Garantir que a classe C++ e o protocolo UDP estejam limpos e modulares para rodar em conjunto com os outros 10 sensores da turma no servidor central.

Se quiser, já podemos começar agora mesmo pela **Fase 0 e Fase 1**: posso gerar o **esboço completo da classe C++ orientada a objetos (`.hpp` e `.cpp`) já documentada no padrão Doxygen** para ler o `/sys/bus/iio/devices/iio:device0/in_voltageX_raw` e o `Makefile` para a toolchain da STM32MP1. Quer que eu monte essa estrutura inicial?

User prompt: eu não vou poder levar o material pra minha casa nem ninguem do grupo , então gostaria de fazer o maximo do trabalho em casa, como faço isso? teria algum simulador?

Response: Como o código do projeto roda em **espaço de usuário Linux** na placa e faz a leitura do sensor lendo um arquivo do sistema (`/sys/bus/iio/devices/iio:device0/in_voltageX_raw`), **você consegue desenvolver e testar 95% de todo o projeto em casa sem precisar da placa física**.

No Linux, "tudo é um arquivo": para o seu programa em C++, ler o conversor A/D da STM32MP1 ou ler um arquivo `.txt` criado no seu PC é exatamente a mesma operação de leitura (`std::ifstream`). Além disso, você pode usar o **QEMU User Mode (`qemu-arm`)** na sua máquina virtual Linux para executar o binário compilado para ARM diretamente no seu PC.

1. Como "Simular" a Placa e o Sensor HW-201 no seu PC

No kit STM32MP1-DK1, o subsistema IIO do Linux expõe o valor bruto do conversor analógico-digital dentro do arquivo `/sys/bus/iio/devices/iio:device0/in_voltageX_raw` (que contém apenas um número inteiro em texto, como `0` quando aterrado no GND ou um valor alto como `4095` / `65535` em 3,3V).

Passo A: Deixe o caminho do arquivo configurável na sua Classe C++

Para cumprir a regra de **zero variáveis globais** e bom encapsulamento POO, passe o caminho do arquivo (e o IP do servidor UDP) no construtor da sua classe ou via argumentos de linha de comando (`argc`, `argv`), deixando o caminho real da placa como padrão:

```
// Exemplo de construtor na sua classe (Entrega 1)
class IrObstacleSensor {
private:
    std::string devicePath;
    int threshold;
    int lastRawValue;

public:
    // Por padrão usa o caminho real da STM32MP1, mas aceita um caminho simulado no PC
    explicit IrObstacleSensor(
        const std::string& path = "/sys/bus/iio/devices/iio:device0/in_voltage0_raw",
        int alertThreshold = 2000
    ) : devicePath(path), threshold(alertThreshold), lastRawValue(0) {}

    int readRaw() {
        std::ifstream file(devicePath);
        if (!file.is_open()) {
```

```
            throw std::runtime_error("Erro ao abrir o canal IIO: " + devicePath);
        }
        file >> lastRawValue;
        return lastRawValue;
    }

    bool isCompartmentViolated() const {
        // No sensor HW-201, obstáculo/reflexão joga a saída para nível BAIXO (0V)
        return lastRawValue < threshold;
    }
};
```

Passo B: Crie um "Simulador do Sensor" no terminal do seu PC

Na sua máquina Linux em casa, crie uma pasta temporária para fingir que é o diretório `/sys/bus/iio/...` da placa e rode um loop no terminal que alterna os valores simulando a carga fechada e sendo violada:

```
# 1. Cria o diretório falso de simulação no seu PC
mkdir -p /tmp/mock_iio

# 2. Simula o sensor normal (sem obstáculo / nível lógico alto)
echo "4095" > /tmp/mock_iio/in_voltage0_raw

# 3. Ou rode um loop automático que simula uma violação a cada 5 segundos:
while true; do
    echo "4095" > /tmp/mock_iio/in_voltage0_raw
    echo "[Simulador] Estado Normal (4095)"
    sleep 5
    echo "120" > /tmp/mock_iio/in_voltage0_raw
    echo "[Simulador] VIOLACAO DETECTADA (120)"
    sleep 5
done
```

Assim, em casa você roda `./meu_programa /tmp/mock_iio/in_voltage0_raw 127.0.0.1`, e no laboratório você roda apenas `./meu_programa` (usando os padrões da placa).

2. Como Testar o Binário Cross-Compilado (ARM) no PC com `qemu-user`

Um erro muito comum é o aluno testar o código compilado com o `g++` normal do PC, achar que está tudo certo, e na hora do laboratório descobrir que a compilação cruzada com a toolchain `arm-buildroot-linux-gnueabihf_sdk-DK2.tar.gz` deu erro.

Você pode **compilar para a placa e rodar o executável ARM dentro do seu próprio PC** usando o emulador de espaço de usuário do QEMU:

1. **Instale a toolchain do PDF na sua VM Linux em casa**:

   ```
   tar -xvf arm-buildroot-linux-gnueabihf_sdk-DK2.tar.gz
   ```

2. **Instale o `qemu-user` na sua VM Linux:**

   ```
   sudo apt update && sudo apt install -y qemu-user qemu-user-static
   ```

3. **Compile seu código para a arquitetura ARM da STM32MP1** usando o `g++` da toolchain:

   ```
   /caminho/da/toolchain/bin/arm-buildroot-linux-gnueabihf-g++ -Wall -O2 main.cpp IrObstacleSensor.cpp -o app_stm32
   ```

4. **Execute o binário ARM da placa direto no seu PC** apontando para o `sysroot` da toolchain:

   ```
   qemu-arm -L /caminho/da/toolchain/arm-buildroot-linux-gnueabihf/sysroot ./app_stm32 /tmp/mock_iio/in_voltage0_ra
   ```

Se rodar sem erros no `qemu-arm` em casa, **é 100% garantido que o binário vai executar de primeira no Linux da STM32MP1-DK1** quando você transferir via `scp` no laboratório.

3. Como Testar o UDP e a Interface Gráfica 100% em Casa

A comunicação entre a placa e o servidor usa **sockets UDP padrão** e a interface gráfica já roda no PC de qualquer forma.

- **Em casa (Modo Localhost):** O seu programa C++ (rodando nativo ou via `qemu-arm`) lê o arquivo `/tmp/mock_iio/in_voltage0_raw` e envia os pacotes UDP para o IP `127.0.0.1` (porta, por exemplo, `5000`). A sua interface gráfica em Python/PyQt escuta na porta `5000` do próprio PC, plota o gráfico dos últimos 30–60 segundos, dispara o alerta visual e salva o CSV.
- **No laboratório (Modo Placa):** Você só muda o IP de destino de `127.0.0.1` para `192.168.42.10` (IP do PC na rede Ethernet com a placa).

📋 Plano de Guerra: O que fazer em Casa vs. No Laboratório

Com essa arquitetura, vocês só precisam de **15 a 20 minutos com a placa no laboratório** por semana para colher evidências e prints obrigatórios:

Entrega	O que fazer 100% em Casa (Sem a Placa)	O que fazer Rápido no Laboratório (Com a Placa)
Entrega 0 (19/10)	Criar o repositório Git e estruturar o `README.md` base.	Conectar os 3 fios do sensor HW-201 na placa **desligada** (Pino 4 = 3V3, Pino 7 = GND, Pino 5 = Sinal) e tirar a foto.
Entrega 1 (26/10)	Escrever toda a classe C++ (`IrObstacleSensor`), comentar com Doxygen, gerar a documentação e testar com o arquivo simulado `/tmp/mock_iio`.	Apenas descobrir qual é o número `X` do arquivo `in_voltageX_raw` (aterrando no GND conforme Figura 3) para deixar como padrão no código.
Entrega 2 (02/11)	Criar o `Makefile` com a toolchain, compilar para ARM, testar com `qemu-arm` e escrever o passo a passo de compilação no `README.md`.	Enviar o binário pronto via `scp` para `root@192.168.42.2`, rodar via SSH, aproximar a mão do sensor e **tirar o print do terminal** para o `README.md`.
Entrega 3 (09/11)	Criar a classe C++ de envio UDP, documentar o protocolo (JSON/CSV) no `README.md` e testar o envio para `127.0.0.1` usando `nc -ul 5000`.	Rodar o binário na placa enviando via cabo Ethernet para `192.168.42.10` e **tirar o print dos pacotes chegando no PC**.
Entrega 4 (16/11)	Desenvolver toda a GUI no PC (gráfico 30–60s, alertas, log CSV, bônus) testando com o simulador em `127.0.0.1`.	Conectar a placa no PC, rodar a GUI recebendo dados reais do sensor e **tirar os prints da interface**.
Entrega 5 (23/11)	Fazer o Fluxograma, Diagrama de Classes UML, explicação do sensor e slides da apresentação no `README.md`.	Gravar o **vídeo de demonstração** com a placa funcionando e participar da simulação integrada.

Quer que eu já monte o **código C++ completo da Entrega 1 e 2** (com suporte a caminho simulado via argumento de terminal) junto com um **script em Python que simula o sensor** para vocês já deixarem tudo rodando em casa hoje?

User prompt: o professor falou pra usar git, mas como eh em grupo, pode usar o github pra ficar online?

Response: **Sim, não só pode como é exatamente o esperado pelo projeto!**

Embora o documento fale em "repositório Git", a **Instrução Geral nº 4** exige que a entrega seja feita enviando o **link do commit final via Google Classroom**. Para que exista um link acessível pelo professor e para que todos os três integrantes do grupo trabalhem juntos, o repositório Git precisa estar hospedado em uma plataforma online — sendo o **GitHub** (ou GitLab) o padrão utilizado.

Além disso, o documento exige que o arquivo `README.md` do repositório exiba capturas de tela (prints) da interface gráfica e do terminal, fluxograma e diagrama de classes, recursos que o GitHub renderiza automaticamente na página inicial do repositório.

💡 Dicas para Configurar o GitHub do Grupo sem Problemas na Avaliação

1. **Visibilidade do Repositório (Público vs. Privado):**
   - Deixe o repositório como **Public (Público)** para garantir que o professor (1º Ten Nicolas Oliveira) consiga abrir o link do commit enviado no Google Classroom sem precisar de convite ou permissão de acesso.
   - Caso queiram deixar privado durante o desenvolvimento, lembrem-se de torná-lo público antes de enviar o link da entrega (ou adicionar a conta do professor como colaboradora).
2. **Adicione os Integrantes do Grupo:**
   - Como a turma é dividida em grupos de três integrantes, uma pessoa cria o repositório no GitHub e vai em **Settings ➔ Collaborators ➔ Add people** para convidar os outros dois colegas. Assim, todos podem dar `git push` e o histórico de commits mostrará a participação de cada um.
3. **Como Pegar o "Link do Commit Final" Exigido na Entrega:**
   - O edital pede especificamente o **link do commit final** de cada etapa (e não apenas o link geral do repositório).
   - No GitHub, após dar o `git push` da entrega da semana, clique em **"X Commits"** (no canto superior direito da lista de arquivos), clique sobre o código/título do último commit daquela entrega e copie a URL do navegador (ela terá o formato `[https://github.com/usuario/nome-do-repo/commit/a1b2c3d4](https://github.com/usuario/nome-do-repo/commit/a1b2c3d4)...`). É esse link que garante ao professor a data e hora exatas da versão entregue.
4. **Organização dos Prints para o `README.md`:**
   - Como todas as evidências e prints das entregas precisam aparecer no `README.md`, criem uma pasta chamada `docs/` ou `assets/` dentro do repositório para salvar as imagens (ex.: a foto da conexão do sensor da Entrega 0 e os prints do terminal e da GUI) e referenciem no Markdown com `![Descrição](docs/foto_entrega0.jpg)`.

User prompt: quero sua ajuda pra saber como posso fazer o entregavel 0, quero um passo a passo. minha ideia ainda eh fazer o projeto, quero aprender e não ficar dependendo mt de IA

Response: A melhor forma de aproveitar esse projeto sem depender de IA é entender **o porquê físico e lógico** de cada conexão antes de colocar a mão na massa. Na **Entrega 0**, seu único objetivo prático é conectar os 3 pinos do sensor infravermelho na placa de forma segura, fotografar e registrar isso no seu repositório Git.

callout-content-1

🧠 Entendendo o Hardware (O "Porquê" da Ligação)

Antes de espetar os fios, vale entender o que está acontecendo nos dois lados:

1. **No lado do seu Sensor Infravermelho (HW-201):**
    - Ao abrir o saquinho antiestático, você verá 3 pinos metálicos na base da plaquinha:
        - **`VCC`:** Recebe a alimentação elétrica para ligar o LED emissor infravermelho e o chip comparador da placa.
        - **`GND`:** É a referência de terra (0 V), que precisa ser compartilhada com a placa STM32MP1.
        - **`OUT` (ou `D0`):** É o pino de saída de dados. Quando o feixe infravermelho reflete em um obstáculo (ou na tampa da carga), o sensor muda a tensão desse pino entre `0 V` e `3,3 V`.
2. **No lado da Placa (`STM32MP157D-DK1`):**
    - Na parte de trás da placa (onde fica o adesivo branco com `USER: root SENHA: root`), existem duas fileiras pretas de conectores fêmea (padrão Arduino).
    - Embora o seu sensor seja digital, o roteiro padroniza a leitura de sensores digitais e analógicos através do conversor Analógico-Digital (ADC) da placa, lido pelo subsistema IIO do Linux (`/sys/bus/iio/devices/iio:device0/`). Por isso, o pino de sinal vai conectado em uma porta analógica da placa.

🛠️ Passo a Passo para Executar a Entrega 0

1 **Preparar a bancada com tudo desligado**

Segurança e materiais

Retire a placa `STM32MP157D-DK1` da caixa e o sensor de obstáculo infravermelho da embalagem. Separe **3 cabos jumper fêmea-macho** (a ponta fêmea entra nos pinos do sensor e a ponta macho entra nos conectores pretos da placa).

Como verificar: Certifique-se de que **nenhum cabo USB** (nem o USB-C de alimentação, nem o Micro-USB serial) esteja conectado à placa.

2 **Conectar os jumpers no sensor HW-201**

Identificação dos 3 pinos

Olhe a serigrafia (texto escrito na placa do sensor) ao lado dos 3 pinos e conecte os jumpers. Se puder, siga o mesmo padrão de cores da **Figura 5** do guia para facilitar a correção visual pelo professor:

- Fio **Vermelho** no pino **`VCC`** do sensor.
- Fio **Marrom (ou Preto)** no pino **`GND`** do sensor.
- Fio **Laranja (ou outra cor viva)** no pino **`OUT` / `D0`** do sensor.

Como verificar: Confira se as três pontas fêmeas estão firmes e se você sabe exatamente qual cor corresponde a `VCC`, `GND` e `SINAL`.

3 **Conectar a alimentação (3V3 e GND) no kit**

Bloco de 8 pinos (Direita)

Posicione a placa com os conectores pretos voltados para você, na mesma orientação da **Figura 4 e Figura 5** do PDF (com o conector do cartão SD à esquerda). Localize o **conector preto de 8 furos à direita** (onde está escrito `IOREF`, `3V3`, `GND`, `VIN` na placa) e conte os furos **da direita para a esquerda (de 1 a 8)**:

- Espete o fio **`VCC` (Vermelho)** no **pino 4** (logo acima da inscrição `3V3` na placa).
- Espete o fio **`GND` (Marrom/Preto)** no **pino 7** (ao lado do pino `VIN`, que é o segundo pino de terra desse conector).

Como verificar: Conte novamente da direita para a esquerda no bloco de 8 furos: os furos 1, 2 e 3 devem estar vazios; o **4** tem o fio `VCC` (`3V3`); os furos 5 e 6 estão vazios; o **7** tem o fio `GND`; e o 8 (`VIN`) está vazio.

4 **Conectar o pino de Sinal no ADC do kit**

Bloco de 6 pinos (Esquerda)

Agora olhe para o **conector preto de 6 furos à esquerda** (entradas analógicas do ADC) e também conte os furos **da direita para a esquerda (de 1 a 6)**:

- Espete o fio **`SINAL (A0/D0)` (Laranja)** no **pino 5** (penúltimo furo da direita para a esquerda).

Como verificar: No bloco de 6 furos, os furos 1, 2, 3 e 4 ficam vazios, o **5** recebe o fio de sinal do sensor, e o 6 (último na ponta esquerda) fica vazio.

5 **Fotografar e subir no GitHub (Entrega 0)**

Documentação e envio

Tire uma foto bem iluminada e focada que mostre claramente a placa desligada, os furos exatos onde os fios entraram no conector da STM32MP1 e os pinos conectados no sensor.

- Crie uma pasta no seu repositório Git (ex.: `docs/` ou `img/`), salve a foto lá e adicione a imagem no seu arquivo `README.md` na seção da **Entrega 0**.
- Faça o `git add`, `git commit` e `git push`, copie o **link desse commit final** no GitHub e envie no Google Classroom (além de mostrar/enviar a foto ao professor antes de ligar a placa).

Como verificar: Abra o repositório no navegador em uma aba anônima para confirmar que o repositório está público e que a foto aparece carregada dentro do `README.md`.

💡 Como Estudar para a Entrega 1 sem Depender de IA

Depois de concluir a Entrega 0, o próximo passo (para o dia 26/10) será criar a **classe em C++**. Para você mesmo construir o código do zero entendendo tudo:

1. Pesquise sobre a biblioteca padrão `<fstream>` do C++ (`std::ifstream`), que é a ferramenta usada para abrir e ler arquivos de texto no Linux.
2. Relembre a estrutura básica de separação em dois arquivos no C++: o arquivo de cabeçalho (`.hpp`), onde você declara a `class`, seus atributos `private` e métodos `public`, e o arquivo de implementação (`.cpp`), onde você escreve a lógica das funções.

User prompt: eu quero um codigo latex completo falando o passo a passo do entregavel 0 e 1, listando o que preciso ter em mãos e o que ja posso ir adiantando e fazendo em casa

Response: O documento LaTeX abaixo reúne o guia passo a passo para a **Entrega 0 (19/10/2026)** e a **Entrega 1 (26/10/2026)** do **Projeto de Programação Aplicada (Monitoramento Inteligente de Carga)**, separando o que você precisa ter em mãos, o que pode ser adiantado em casa e o que deve ser feito no laboratório com o kit **STM32MP157D-DK1** e o **Sensor de Obstáculo Infravermelho Reflexivo Digital (HW-201)**.

Resumo das Diretrizes do Roteiro Oficial Incluídas no Documento

- **Segurança de Hardware (Entrega 0):** É obrigatório enviar ao professor a foto da conexão do sensor com o kit **desligado** antes de energizá-lo. A alimentação do sensor deve ser feita exclusivamente em **3,3V** (pino 4 do conector de alimentação), o terra no **GND** (pino 7) e o sinal no pino **5** do conector ADC, sem jamais utilizar 5V ou remover o cartão microSD.
- **Requisitos de Software (Entrega 1 — 2,0 pontos):** Implementação de uma classe em C++ para leitura do sensor documentada com **Doxygen**, respeitando os princípios de Programação Orientada a Objetos (POO), **sem variáveis globais** e sem funções soltas fora de classes.
- **Mapeamento no Linux Embarcado:** A leitura na placa ocorre via subsistema IIO no diretório `/sys/bus/iio/devices/iio:device0/in_voltageX_raw`, descobrindo o canal correto ao conectar o pino do ADC ao GND e verificar qual arquivo retorna zero com o comando `cat`.
- **Repositório Git e `README.md`:** Todas as entregas devem ser submetidas via link do commit final no Google Classroom, mantendo o `README.md` atualizado com descrição, dependências, estrutura das classes, instruções de compilação/execução e evidências.

Código LaTeX Completo (Pronto para Overleaf / TeX Live)

```
\documentclass[11pt,a4paper]{article}

% ==================== PACOTES ====================
\usepackage[utf8]{inputenc}
\usepackage[T1]{fontenc}
\usepackage[brazil]{babel}
\usepackage{geometry}
\geometry{top=2.2cm, bottom=2.2cm, left=2.2cm, right=2.2cm}
\usepackage{hyperref}
\usepackage{enumitem}
\usepackage{booktabs}
\usepackage{xcolor}
\usepackage{listings}
\usepackage{tcolorbox}
\tcbuselibrary{skins,breakable}

% ==================== CONFIGURAÇÃO DE CORES E ESTILOS ====================
\definecolor{imeblue}{RGB}{0, 51, 102}
\definecolor{alertred}{RGB}{180, 20, 20}
\definecolor{homegreen}{RGB}{20, 110, 50}
\definecolor{codebg}{RGB}{245, 247, 250}

\hypersetup{
    colorlinks=true,
    linkcolor=imeblue,
    urlcolor=imeblue
}

\lstset{
    backgroundcolor=\color{codebg},
    basicstyle=\ttfamily\small,
    breaklines=true,
```

```
    frame=single,
    rulecolor=\color{gray!40},
    showstringspaces=false,
    commentstyle=\color{homegreen},
    keywordstyle=\color{imeblue}\bfseries
}

% Caixas personalizadas
\newtcolorbox{alertbox}[1]{
    colback=alertred!5!white,
    colframe=alertred,
    fonttitle=\bfseries,
    title=#1,
    breakable
}

\newtcolorbox{homebox}[1]{
    colback=homegreen!5!white,
    colframe=homegreen,
    fonttitle=\bfseries,
    title=#1,
    breakable
}

\newtcolorbox{labbox}[1]{
    colback=imeblue!5!white,
    colframe=imeblue,
    fonttitle=\bfseries,
    title=#1,
    breakable
}

% ==================== DOCUMENTO ====================
\begin{document}

\begin{center}
    {\Large \textbf{INSTITUTO MILITAR DE ENGENHARIA (IME)}}\\[0.15cm] %[cite: 4]
    {\large Projeto de Programação Aplicada --- Monitoramento Inteligente de Carga}\\[0.1cm] %[cite: 4]
    {\normalsize \textbf{Guia de Execução e Estudo: Entregáveis 0 e 1}}\\[0.2cm]
    \begin{tabular}{ll}
        \toprule
        \textbf{Kit de Desenvolvimento:} & STMicroelectronics STM32MP157D-DK1 (Linux Embarcado) \\ %[cite: 2, 3, 6]
        \textbf{Sensor do Grupo (Nº 3):} & Módulo de Obstáculo Infravermelho Reflexivo Digital (HW-201 / SKU: SS10)
        \textbf{Entrega 0:} & 19/10/2026 (Foto da conexão física com o kit desligado) \\ %[cite: 8, 10]
        \textbf{Entrega 1:} & 26/10/2026 (Classe C++ de leitura + Doxygen --- 2,0 pontos) \\ %[cite: 8]
        \bottomrule
    \end{tabular}
\end{center}

\vspace{0.3cm}

% ======================================================================
\section{O Que Preciso Ter em Mãos (Checklist de Materiais)}
% ======================================================================

Para otimizar o tempo de laboratório e adiantar o máximo do projeto em casa, separe os recursos abaixo:

\subsection{No Laboratório (Acesso Físico ao Hardware)}
\begin{itemize}[leftmargin=*]
    \item \textbf{Placa STM32MP157D-DK1} com o cartão microSD original inserido no slot. %[cite: 2, 3, 9, 10]
    \item \textbf{Módulo Sensor Infravermelho Reflexivo HW-201} (3 pinos: \texttt{VCC}, \texttt{GND} e \texttt{OUT/D
    \item \textbf{3 cabos jumper Fêmea-Macho} (preferencialmente nas cores \textbf{Vermelho}, \textbf{Marrom/Preto}
    \item \textbf{1 cabo jumper Macho-Macho} (apenas para a Entrega 1, para aterrar o pino ADC no \texttt{GND} e des
    \item \textbf{Cabo USB-C} (alimentação na porta \texttt{PWR\_IN}) e \textbf{Cabo Micro-USB} (acesso serial via S
    \item \textbf{Cabo de rede Ethernet (RJ45)} para conexão via SSH/SCP. %[cite: 9, 10]
    \item \textbf{Celular/Câmera} para fotografar a conexão física nítida da Entrega 0. %[cite: 8, 10]
\end{itemize}

\subsection{Em Casa (Ambiente de Desenvolvimento no PC)}
\begin{itemize}[leftmargin=*]
    \item \textbf{Ambiente Linux} (Máquina Virtual no VirtualBox conforme indicado no guia, dual-boot ou WSL2). %[ci
    \item \textbf{Pacote de ferramentas básicas no Linux:} compilador \texttt{g++}, \texttt{make}, \texttt{git} e \t
    \item \textbf{Toolchain de Compilação Cruzada:} arquivo \texttt{arm-buildroot-linux-gnueabihf\_sdk-DK2.tar.gz} b
    \item \textbf{Conta no GitHub:} repositório criado e compartilhado com os 3 integrantes do grupo. %[cite: 4, 7]
\end{itemize}

% ======================================================================
\section{Entregável 0 --- Conexão Física e Repositório (19/10/2026)}
% ======================================================================
```

```
\begin{alertbox}{Regras Críticas de Segurança (Leitura Obrigatória)}
\begin{enumerate}[leftmargin=*]
    \item \textbf{Kit Totalmente Desligado:} É obrigatório enviar ao professor a foto mostrando a conexão do sensor
    \item \textbf{Proibido usar 5V:} Os sensores devem ser alimentados sempre com \textbf{3,3V}. Jamais conecte no p
    \item \textbf{Cartão SD:} Não retire o cartão microSD presente no kit para não corromper ou perder a imagem do L
\end{enumerate}
\end{alertbox}

\subsection{O que já posso adiantar em CASA para a Entrega 0}
\begin{homebox}{Tarefas em Casa --- Entrega 0}
\begin{enumerate}[leftmargin=*]
    \item \textbf{Criar o Repositório no GitHub:}
    \begin{itemize}
        \item Crie um repositório \textbf{Público} e adicione os outros dois integrantes do grupo em \textit{Setting
        \item Crie a estrutura inicial de pastas do projeto: \texttt{src/} (implementação \texttt{.cpp}), \texttt{in
    \end{itemize}
    \item \textbf{Escrever a Base do \texttt{README.md}:}
    \begin{itemize}
        \item O guia exige que o \texttt{README.md} seja atualizado a cada etapa. %[cite: 7]
        \item Escreva já em casa: (a) \textbf{Descrição Geral do Projeto} (monitoramento de abertura/violação de com
    \end{itemize}
\end{enumerate}
\end{homebox}

\subsection{Passo a Passo no LABORATÓRIO para a Entrega 0}
\begin{labbox}{Tarefas no Laboratório --- Entrega 0 (10 minutos)}
Com a placa \textbf{desconectada de qualquer cabo USB}, vire-a com os conectores pretos (padrão Arduino) voltados pa

\begin{center}
\begin{tabular}{llll}
\toprule
\textbf{Pino no Sensor HW-201} & \textbf{Cor Sugerida} & \textbf{Conector na STM32MP1} & \textbf{Posição Exata (Dire
\midrule
\texttt{VCC} (Alimentação) & Vermelho & Bloco da Direita (8 furos) & \textbf{Pino 4} (serigrafia \texttt{3V3}) \\ %[
\texttt{GND} (Terra) & Marrom/Preto & Bloco da Direita (8 furos) & \textbf{Pino 7} (serigrafia \texttt{GND}, ao lado
\texttt{OUT / D0} (Sinal) & Laranja & Bloco da Esquerda (6 furos) & \textbf{Pino 5} (penúltimo furo à esquerda) \\ %
\bottomrule
\end{tabular}
\label{tab:pinagem}
\end{center}

\textbf{Conferência Final e Envio:}
\begin{enumerate}[leftmargin=*]
    \item Verifique duas vezes a contagem dos furos da direita para a esquerda em ambos os blocos. %[cite: 12]
    \item Tire uma foto bem focada mostrando simultaneamente os 3 fios no sensor, os furos nos conectores da placa e
    \item Salve a foto na pasta \texttt{docs/} do repositório, referencie no \texttt{README.md}, faça \texttt{git co
\end{enumerate}
\end{labbox}

% ====================================================================
\section{Entregável 1 --- Classe C++ e Doxygen (26/10/2026 --- 2,0 pts)}
% ====================================================================

\subsection{Requisitos Técnicos Exigidos na Avaliação}
\begin{itemize}[leftmargin=*]
    \item \textbf{Programação Orientada a Objetos (POO) Estrita:} É expressamente \textbf{proibida a utilização de v
    \item \textbf{Leitura em Espaço de Usuário Linux:} O sensor conectado na porta ADC é lido através do subsistema
    \item \textbf{Documentação Doxygen:} Todo o código da classe deve estar documentado no padrão Doxygen e bem enca
\end{itemize}

\subsection{O que posso fazer 100\% em CASA para a Entrega 1}
\begin{homebox}{Tarefas em Casa --- Entrega 1 (Desenvolvimento e Simulação)}

\textbf{Passo 1: Simular o Hardware no seu PC Linux}\\
Como o Linux embarcado expõe o conversor A/D como um arquivo de texto contendo um número inteiro (ex.: \texttt{0} pa
\begin{lstlisting}[language=bash]
mkdir -p /tmp/mock_iio
echo "4095" > /tmp/mock_iio/in_voltage0_raw
\end{lstlisting}

\textbf{Passo 2: Projetar e Programar a sua Classe C++ (Sem depender de IA)}\\
Divida seu código em um arquivo de cabeçalho (\texttt{include/IrObstacleSensor.hpp}) e um de implementação (\texttt{
\begin{itemize}
    \item \textbf{Atributos Privados (\texttt{private}):}
    \begin{itemize}
        \item \texttt{std::string devicePath}: guarda o caminho do arquivo IIO (permitindo passar o caminho simulado
        \item \texttt{int lastRawValue}: armazena a última leitura bruta obtida do arquivo. %[cite: 6, 11]
        \item \texttt{int threshold}: limiar de decisão entre nível lógico baixo (obstáculo/violação detectada) e ní
```

```
    \end{itemize}
    \item \textbf{Métodos Públicos (\texttt{public}):}
    \begin{itemize}
        \item \textbf{Construtor:} inicializa os atributos privados usando lista de inicialização.
        \item \textbf{Método de Leitura Bruta (ex.: \texttt{readRaw()}):} utiliza a biblioteca \texttt{<fstream>} (\
        \item \textbf{Método de Interpretação Física (ex.: \texttt{isViolated()}):} avalia se a leitura indica abert
        \item \textbf{Getters e Setters:} métodos \texttt{const} para consultar o último valor lido ou alterar o lim
    \end{itemize}
\end{itemize}

\textbf{Passo 3: Documentar com Doxygen}\\
Acima da declaração da classe, dos atributos e de cada método no arquivo \texttt{.hpp}, escreva os blocos de documen
\begin{lstlisting}[language=C++]
/**
 * @file IrObstacleSensor.hpp
 * @brief Classe para leitura do sensor infravermelho reflexivo HW-201 via IIO.
 */

/**
 * @class IrObstacleSensor
 * @brief Gerencia a aquisicao e interpretacao do sensor de obstaculo IR.
 */
class IrObstacleSensor {
    // ...
    /**
     * @brief Realiza a leitura bruta do canal ADC no sysfs do Linux.
     * @return Valor inteiro correspondente a leitura bruta do conversor A/D.
     */
    int readRaw();
};
\end{lstlisting}
Para gerar a documentação em casa e testar, execute no terminal:
\begin{lstlisting}[language=bash]
doxygen -g Doxyfile   # Gera o arquivo de configuracao padrao
doxygen Doxyfile      # Gera a pasta html/ e latex/ com a documentacao
\end{lstlisting}

\textbf{Passo 4: Preparar a Toolchain da Entrega 2}\\
Já aproveite para extrair a toolchain fornecida pelo professor na sua máquina Linux: %[cite: 8]
\begin{lstlisting}[language=bash]
tar -xvf arm-buildroot-linux-gnueabihf_sdk-DK2.tar.gz
\end{lstlisting}
\end{homebox}

\subsection{O que fazer no LABORATÓRIO para a Entrega 1}
\begin{labbox}{Tarefas no Laboratório --- Entrega 1 (Descoberta do Canal ADC)}
Após o professor aprovar a foto da Entrega 0, descubra qual é o número \texttt{X} exato do arquivo \texttt{in\_volta

\begin{enumerate}[leftmargin=*]
    \item Com a placa desligada, coloque um jumper ligando o \textbf{Pino 5 do conector ADC (6 furos)} diretamente a
    \item Conecte o cabo USB-C na porta \texttt{PWR\_IN} e o cabo Micro-USB na porta Serial. %[cite: 9]
    \item Abra o terminal serial (TeraTerm, \texttt{minicom} ou PuTTY) configurado para \textbf{Baudrate 115200, 8N1
    \item Navegue até o diretório IIO e teste os canais disponíveis: %[cite: 11]
\begin{lstlisting}[language=bash]
cd /sys/bus/iio/devices/iio:device0/
ls
cat in_voltage*_raw
\end{lstlisting}
    \item O arquivo \texttt{in\_voltageX\_raw} que retornar o valor \textbf{zero} (ou muito próximo de zero) é o can
    \item Atualize o caminho padrão na sua classe C++, revise o \texttt{README.md} com a explicação da classe e de s
\end{enumerate}
\end{labbox}

\end{document}
```

Como Compilar este Código

1. Acesse o **Overleaf** (ou seu editor LaTeX local), crie um projeto em branco (`Blank Project`) e cole todo o bloco acima no arquivo `main.tex`.
2. Clique em **Recompile** — o documento utiliza apenas pacotes nativos do TeX Live (`tcolorbox`, `listings`, `booktabs`, `geometry`) e não exige arquivos externos para compilar.
